

RESUMEN EXPRESS · UNA NOCHE Y UN DÍA

Solo fórmulas y teoría

Cada fórmula con lo que significa cada símbolo, la idea clave de cada cuaderno y lo que no puedes olvidar. Del perceptrón a las convoluciones.



Dos bloques: teoría base hoy, redes e imágenes mañana

Esta noche • teoría base

Perceptrón y su entrenamiento, preprocesamiento y métricas, tensores, softmax, derivadas y descenso del gradiente, Keras. Diapositivas 3 a 9. Al terminar, di en voz alta qué significa cada símbolo sin mirar.

Mañana • redes e imágenes

Activaciones, pérdidas, batch y overfitting, imágenes, convolución, CNN. Diapositivas 10 a 14. Cierra con la hoja de repaso (15) y el cuestionario de práctica.

Truco: no memorices texto. Para cada fórmula pregúntate: ¿qué entra, qué sale y para qué sirve?

Suma ponderada más sesgo, y un escalón que decide

$$Z = W_1X_1 + W_2X_2 + \dots + b$$

$$\hat{y} = 1 \text{ si } z > 0 \cdot \hat{y} = 0 \text{ si no}$$

El perceptrón es la neurona más simple: pondera las entradas, las suma y decide entre dos clases. Base de las redes modernas (varias capas = MLP).

- Solo separa clases con una **recta** (hiperplano): problemas
- linealmente separables. No resuelve XOR.
-

Término	Qué significa
x (entradas)	Características del dato (ej. nota, promedio)
w (pesos)	Importancia de cada característica; es lo que se aprende
b (sesgo, bias)	Umbral: desplaza la frontera de decisión
z (suma ponderada)	Combinación lineal de entradas y pesos
Escalón	Función de activación: convierte z en 0 o 1
$\hat{y}$	Predicción del modelo

El perceptrón corrige sus pesos en proporción al error

$$\begin{aligned}\text{error} &= y - \hat{y} \\ w &\leftarrow w + \eta \cdot \text{error} \cdot x \\ b &\leftarrow b + \eta \cdot \text{error}\end{aligned}$$

Si acierta, error = 0 y nada cambia. Si debía ser 1 y dijo 0, los pesos suben; si al revés, bajan. Se repite por cada dato y por cada época hasta que el error llega a 0 o se acaban las épocas.

sklearn: **Perceptron(max_iter, eta0, tol, random_state)**; se lee **coef_** (pesos), **intercept_** (sesgo), **n_iter_** (épocas). Para cuando la mejora es menor que tol varias épocas seguidas.

Término	Qué significa
y	Valor real (etiqueta)
η (tasa de aprendizaje)	Tamaño del paso de ajuste. Alta: oscila o no converge. Baja: lento
Época	Una pasada completa por todos los datos
Underfitting	Pocas épocas o modelo simple: no aprende
Overfitting	Memoriza el entrenamiento y falla en datos nuevos
Early stopping	Parar cuando la validación deja de mejorar

Escalar bien y medir con la métrica correcta

MinMax: $x' = (x - \min) / (\max - \min)$

Estándar: $z = (x - \mu) / \sigma$

MinMax deja todo entre 0 y 1; el estándar deja media 0 y desviación 1 (μ = media, σ = desviación).

- El escalador se ajusta con **fit_transform en train** y solo
- J transform en test**. Si no, hay fuga de datos (data leakage).
- stratify=y mantiene la proporción de clases.

Accuracy = $(VP + VN) / \text{total}$

Precision = $VP / (VP + FP)$

Recall = $VP / (VP + FN)$

Término	Qué significa
VP / VN	Aciertos positivos / negativos
FP	Dice positivo y era negativo (falsa alarma)
FN	Dice negativo y era positivo (se le escapa)
Recall	De los positivos reales, cuántos detecta. Clave en medicina: un FN es un enfermo sin tratar

Tensores, ejes y la diferencia entre constante y variable

Término	Qué significa
Tensor	Arreglo multidimensional (como un array de NumPy) que puede ir a la GPU
shape	Tamaño en cada eje, ej. (2, 3) = 2 filas, 3 columnas
dtype	Tipo de dato: float32, int32...
tf.constant	Tensor inmutable
tf.Variable	Valor modificable (los pesos): assign, assign_add, assign_sub
@tf.function	Convierte la función en grafo optimizado; traza solo la primera vez

`axis=0` → un resultado por columna
`axis=1` → un resultado por fila

`reduce_mean` y `reduce_sum` reducen un eje. `concat axis=0` apila filas (más alto); `axis=1` pega columnas (más ancho). `@` es producto matricial; `transpose` cambia filas por columnas.

Keras = alto nivel, rápido para prototipar. **TensorFlow** = bajo nivel, calcula gradientes solo. **PyTorch** = grafos dinámicos. **Scikit-learn** = ML clásico.

Softmax, sigmoide y qué dice una derivada

$$\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

Convierte un vector en probabilidades que suman 1. Pasos: exponencial de cada valor, sumar, dividir. Salida multiclase.

$$\sigma(x) = 1 / (1 + e^{-x})$$

Sigmoide: un solo valor entre 0 y 1. $\sigma(0) = 0.5$. Salida binaria.

$f'(x) \rightarrow$ pendiente
 $f''(x) \rightarrow$ curvatura

Término	Qué significa
$f' = 0$	Punto crítico (posible mínimo o máximo)
$f'' > 0$	Cóncava hacia arriba $\rightarrow$ mínimo
$f'' < 0$	Cóncava hacia abajo $\rightarrow$ máximo
GradientTape	Graba las operaciones y deriva hacia atrás (regla de la cadena)
Dos cintas	Anidadas dan la segunda derivada

Moverse contra la pendiente para bajar el error

$$\text{MSE} = (1/n) \sum (\hat{y}_i - y_i)^2$$

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

Idea: la derivada de la pérdida dice hacia dónde sube el error; se da un paso en sentido **contrario**. Repetir hasta llegar al fondo de la curva (mínimo).

- Signo: el perceptrón suma ($+\eta \cdot \text{error} \cdot x$) porque $\text{error} = y - \hat{y}$;
- el descenso del gradiente resta ($-\eta \cdot \text{gradiente}$).
-

Término	Qué significa
L (pérdida)	Número que mide qué tan mal predice el modelo
$\partial L / \partial w$ (gradiente)	Cuánto cambia la pérdida si mueves w un poquito
$\hat{y} = w_1x + w_2x^2 + b$	Modelo del cuaderno: una neurona sin activación con entradas x y x^2
assign_sub	Resta el paso al peso (actualización manual)
SGD / Adam	Optimizadores: aplican la actualización automáticamente

Declarar, compilar, entrenar y evaluar una red

```
parámetros Dense = entradas × neuronas
+ neuronas
```

Cada neurona tiene un peso por entrada más un sesgo. Dropout, Flatten y Pooling no tienen parámetros.

Forma	Cómo se ve
Sequential lista	Sequential([Input, Dense, ...])
.add()	m.add(Dense(...)) capa por capa
API funcional	x = Dense(...)(x); Model(inputs, outputs)

La funcional es la más flexible: varias entradas o salidas, ramas.

Término	Qué significa
compile()	Define optimizer, loss y metrics antes de entrenar
fit()	Entrena: forward, pérdida, backward, actualizar pesos
epochs / batch_size	Pasadas completas / datos antes de cada actualización
validation_data	Datos para medir en cada época sin entrenar con ellos
verbose	0 nada, 1 barra, 2 una línea por época
evaluate / predict	Pérdida y métrica / predicciones nuevas
history.history	loss, val_loss, accuracy por época

Las activaciones dan la no linealidad

Función	Fórmula	Rango	Dónde se usa
ReLU	$\max(0, x)$	$[0, \infty)$	Capas ocultas, opción por defecto
Sigmoid	$1 / (1 + e^{-x})$	$(0, 1)$	Salida binaria

OJ Dense sin activation = **linear**. Varias capas lineales equivalen a una sola: la red no aprende nada no lineal. Por eso las ocultas llevan ReLU.

v
a
n
i
s
h
i
n
g
r
a
d
i
e
n
t
:
s
i
g
m
o
i
d
y
t
a

La pérdida depende del problema y de la salida

Problema	Salida	Pérdida (loss=)
Regresión	1 neurona linear	'mse' o 'mae'
Binaria	1 neurona sigmoid	'binary_crossentropy'
Multiclase, etiquetas one-hot	N neuronas softmax	'categorical_crossentropy'
Multiclase, etiquetas enteras	N neuronas softmax	'sparse_categorical_crossentropy'

$$\text{MSE} = (1/n) \sum (y - \hat{y})^2$$

$$\text{MAE} = (1/n) \sum |y - \hat{y}|$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$\text{BCE} = -[y \log p + (1-y) \log(1-p)]$$

MSE castiga más los errores grandes (sensible a outliers). **MAE** es el error promedio directo. **RMSE** vuelve a las unidades originales. **BCE** castiga mucho la confianza equivocada.

Pérdida: se minimiza al entrenar y debe ser diferenciable. **Métrica:** solo evalúa (accuracy, recall...).

Batch, división de datos y cómo frenar el sobreajuste

lotes por época = $\lceil n / \text{batch} \rceil$

Batch = datos procesados antes de actualizar pesos. Mini-batch (16, 32, 64, 128) es el punto medio entre todo el dataset y una muestra.

Conjunto	Para qué
Train	Aprende los pesos
Validation	Ajusta hiperparámetros, detecta overfitting
Test	Evaluación final, nunca para ajustar

Overfitting: $\text{loss} \downarrow$ y $\text{val_loss} \uparrow$

Término	Qué significa
patience	Épocas sin mejora antes de parar
min_delta	Mejora mínima que cuenta
restore_best_weights	Vuelve a la mejor época
Dropout	Apaga neuronas al azar al entrenar
L2	Penaliza pesos grandes

Círculos concéntricos: una neurona no puede (no son linealmente separables); con capas ocultas ReLU sí.

Una imagen es una matriz de píxeles

Término	Qué significa
Píxel	Número de intensidad, 0 a 255 en uint8
Gris	Matriz 2D (alto, ancho)
Binaria	Solo dos valores (blanco y negro)
Color	3D (alto, ancho, 3 canales)
flatten / ravel	Aplana a 1D (copia / vista); las densas lo necesitan
Umbral	> umbral → 255, si no → 0

$$\text{Gris} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

El ojo es más sensible al verde, luego al rojo y menos al azul.

O OpenCV lee en **BGR**, no RGB: usar `cv.cvtColor(img, COLOR_BGR2RGB)`. Si confundes el orden, el gris sale mal.
J
O

Otsu: elige el umbral que maximiza la varianza entre clases. **K-Means:** agrupa píxeles por color para segmentar.

La CNN aprende sus propios kernels

Convolución: poner el kernel sobre una vecindad, multiplicar elemento a elemento y sumar → nuevo píxel.

Kernel	Efecto
Promedio (1/9)	Desenfoca
Gaussiano	Desenfoque suave, más peso al centro
Mediana	Quita ruido, conserva bordes
Sobel	Detecta bordes: $\sqrt{Sx^2 + Sy^2}$
Nitidez	Realza detalles

Si el kernel suma 1, conserva el brillo.

$$\text{salida} = (N - K + 2P) / S + 1$$

$$\text{parám. Conv2D} = (K \cdot K \cdot C + 1) \cdot \text{filtros}$$

Término	Qué significa
N, K, P, S	Entrada, kernel, padding, stride
C	Canales de entrada (3 en RGB)
MaxPooling 2×2	Divide alto y ancho entre 2, sin parámetros
Flatten	Pasa de 3D a vector para las Dense
from_logits=True	La última Dense no tiene softmax

Catorce ideas que no puedes olvidar

01 Perceptrón = recta; no resuelve XOR ni círculos

02 η alta oscila, η baja es lenta

03 Escalar con fit en train, transform en test

04 Recall = $VP/(VP+FN)$, clave cuando un FN es grave

05 axis=0 por columna, axis=1 por fila

06 Softmax suma 1; sigmoide da un valor 0–1

07 Gradiente: $w \leftarrow w - \eta \cdot \partial L / \partial w$

08 Dense = entradas · neuronas + neuronas

09 Sin activación = linear; ocultas con ReLU

10 Enteros → sparse; one-hot → categorical

11 Overfitting: loss ↓ y val_loss ↑

12 Lotes = $\lceil n / \text{batch} \rceil$, se redondea hacia arriba

13 OpenCV lee BGR

14 Conv: $(N-K+2P)/S+1 \cdot (K \cdot K \cdot C+1) \cdot \text{filtros}$